POURQUOI SQLITE POUR CE PROJET ?

Zéro installation

Pas de serveur MySQL/Postgres à démarrer. La BDD = un seul fichier.

Portable

On peut partager le fichier `.db` dans le zip du projet, avec les données déjà chargées.

Natif CI4

CI4 supporte SQLite3 nativement via PDO. Aucune lib à installer.

Query Builder intact

Tout le code CI4 (Model, Query Builder, migrations) fonctionne sans changer une ligne.

ÉTAPE 1 — VÉRIFIER QUE PDO SQLITE EST ACTIVÉ

1 **Vérifier l'extension PHP**

Dans le terminal, à la racine du projet

```
$ php -m | grep -i sqlite          doit afficher "sqlite3" et "pdo_sqlite"
```

Si rien ne s'affiche, ouvrir `php.ini` et décommenter `extension=pdo_sqlite` et `extension=sqlite3`. Redémarrer le serveur PHP.

Avec XAMPP/WAMP sur Windows, l'extension est souvent déjà présente. Sur macOS avec Homebrew : `brew install php` l'inclut par défaut.

ÉTAPE 2 — CONFIGURER DATABASE.PHP

2 **Modifier `app/Config/Database.php`**

Remplacer la configuration MySQL/Postgres par défaut

Avant (MySQL habituel)

```
'DSN'      => '',
'hostname' => 'localhost',
'username' => 'root',
'password' => '',
'database' => 'fitspace',
'DBDriver' => 'MySQLi',
'DBPrefix' => '',
'port'     => 3306,
```

Après (SQLite)

```
'DSN'      => '',
'hostname' => '',
'username' => '',
'password' => '',
'database' => WRITEPATH.'fitspace.db',
'DBDriver' => 'SQLite3',
'DBPrefix' => '',
'port'     => '',
```

`WRITEPATH` est une constante CI4 qui pointe vers `writable/`. Le fichier `fitspace.db` sera créé automatiquement à la première migration.

3 Différences SQL à connaître

SQLite a quelques particularités par rapport à MySQL/Postgres

MySQL / Postgres

```
INT AUTO_INCREMENT
VARCHAR(255)
TINYINT(1)
ENUM('a','b','c')
NOW()
```

SQLite (CI4 Forge)

```
INTEGER (auto avec PK)
VARCHAR(255) ✓ identique
INTEGER (0 ou 1)
VARCHAR(20) + contrainte check
datetime (valeur PHP)
```

Exemple de migration correcte pour la table `users` :

```php
// app/Database/Migrations/2025-06-01-000001_CreateUsers.php
public function up(): void
{
    $this->forge->addField([
        'id'         => ['type' => 'INTEGER', 'auto_increment' => true],
        'nom'        => ['type' => 'VARCHAR', 'constraint' => 100],
        'email'      => ['type' => 'VARCHAR', 'constraint' => 150],
        'password'   => ['type' => 'VARCHAR', 'constraint' => 255],
        'role'       => ['type' => 'VARCHAR', 'constraint' => 10,
                         'default' => 'client'],   // 'client' ou 'admin'
        'created_at' => ['type' => 'DATETIME', 'null' => true],
    ]);
    $this->forge->addKey('id', true); // true = PRIMARY KEY
    $this->forge->createTable('users');
}
```

Clé étrangère dans `reservations` :

```php
'user_id'    => ['type' => 'INTEGER'],
'creneau_id' => ['type' => 'INTEGER'],
'statut'     => ['type' => 'VARCHAR', 'constraint' => 15,
                 'default' => 'en_attente'],

// Après addField() :
$this->forge->addForeignKey('user_id',    'users',    'id');
$this->forge->addForeignKey('creneau_id', 'creneaux', 'id');
```

SQLite ne vérifie pas les FK par défaut. Ajouter PRAGMA `foreign_keys = ON;` dans le seeder ou via un event CI4 si la vérification est nécessaire en TD.

ÉTAPE 4 — SEEDER : INSÉRER LES DONNÉES DE TEST

4 **Seeder avec données initiales**
app/Database/Seeds/FitspaceSeeder.php

```php
public function run(): void
{
    // Compte admin (mot de passe : admin123)
    $this->db->table('users')->insert([
        'nom'        => 'Administrateur',
        'email'      => 'admin@fitspace.mg',
        'password'   => password_hash('admin123', PASSWORD_DEFAULT),
        'role'       => 'admin',
        'created_at' => date('Y-m-d H:i:s'),
    ]);

    // Ressource de test
    $this->db->table('ressources')->insert([
        'nom'      => 'Salle Zen',
        'type'     => 'cours',
        'capacite' => 10,
    ]);

    // Créneau de test (date dynamique)
    $this->db->table('creneaux')->insert([
        'ressource_id'  => 1,
        'date_debut'    => date('Y-m-d', strtotime('+1 day')).' 08:00:00',
        'date_fin'      => date('Y-m-d', strtotime('+1 day')).' 09:30:00',
        'places_dispo'  => 10,
        'actif'         => 1,
    ]);
}
```

ÉTAPE 5 — LANCER L'INSTALLATION (3 COMMANDES)

5 **Commandes à la racine du projet**
Ordre important à respecter

```
$ php spark migrate                    crée le fichier writable/fitspace.db et les tables
$ php spark db:seed FitspaceSeeder     insère admin + données de test
$ php spark serve                      démarre le serveur sur localhost:8080
```

- C'est tout. Pas de pgAdmin, pas de phpMyAdmin, pas de mot de passe serveur. Le projet tourne immédiatement.

RÉCAPITULATIF DES PIÈGES COURANTS

- **Permissions :** le dossier `writable/` doit être accessible en écriture par le serveur web. Sous Linux : `chmod -R 777 writable/`
- **ALTER TABLE :** SQLite ne supporte pas `ALTER TABLE ... DROP COLUMN` avant SQLite 3.35. Toujours passer par une nouvelle migration, jamais modifier une migration existante.
- **ENUM inexistant :** utiliser VARCHAR avec des valeurs contrôlées côté PHP. Ne pas écrire `ENUM('client','admin')` dans les migrations.
- **Remettre à zéro :** pour repartir de zéro, supprimer le fichier `writable/fitspace.db` et relancer `php spark migrate`. Équivalent du `DROP DATABASE`.